

Standard library hardening

Document #: P3471R2
Date: 2024-12-14
Project: Programming Language C++
Audience: Library Evolution, SG23
Reply-to: Konstantin Varlamov
<varconst@apple.com>
Louis Dionne
<ldionne@apple.com>

§ 1 Introduction

This paper proposes introducing *standard library hardening* into the C++ Standard. Hardening allows turning some instances of undefined behavior in the standard library into a contract violation. This proposal is based on our experience implementing [hardening in libc++](#) and deploying it widely.

§ 2 Revision history

- R0: Initial proposal based on “terminating in an implementation-defined way”.
- R1: Revision rebased on top of contract violations.
- R2: Added full wording and an FAQ section.

§ 3 Motivation

There has been significantly increased attention to safety and security in C++ over the last few years, as exemplified by the well-known [White House report](#) and numerous recent security-related proposals.

While it is important to explore ways to make *new* code safer, we believe that the highest priority to deliver immediate real-world value should be to make *existing* code safer with minimal or no effort on behalf of users. Indeed, the amount of existing security-critical C++ code is so large that rewriting it or modifying it is both economically unviable and dangerous given the risk of introducing new issues.

There have been a few proposals accepted recently that eliminate some cases of undefined behavior in the core language. The standard library also contains many instances of undefined behavior, some of which is a direct source of security vulnerabilities; addressing

those is often trivial, can be done with low overhead and almost no work on behalf of users.

In fact, at the moment all three major library implementations have some notion of a hardened or debug mode. This clearly shows interest, both from users and from implementers, in having a safer mode for the standard library. However, we believe these efforts would be vastly more useful if they were standardized and provided portable, cross-platform guarantees to users; as it stands, implementations differ in levels of coverage, performance guarantees and ways to enable the safer mode.

Finally, leaving security of the library to be a pure vendor extension fails to position ISO C++ as providing a credible solution for code bases with formal security requirements. We believe that formally requiring the basic safety guarantees that most implementations already provide in one way or another could make a significant difference from the point of view of anyone writing or following safety and security coding standards and guidelines.

§ 4 Deployment experience

All three major implementations provide vendor-specific ways of enabling library assertions as proposed in this paper, today.

- We have experience deploying hardening on Apple platforms in several existing codebases.
- Google recently published [an article](#) where they describe their experience with deploying this very technology to hundreds of millions of lines of code. They reported a performance impact as low as 0.3% and finding over 1000 bugs, including security-critical ones.
- Google Andromeda published [an article](#) ~1 year ago about their successful experience enabling hardening.
- The libc++ maintainers have received numerous informal reports of hardening being turned on and helping find bugs in codebases.

Overall, standard library hardening has been a huge success, in fact we never expected so much success. The reception has been overwhelmingly positive and while the quality of implementation will never be perfect, we are working hard to expand the scope of hardening in libc++, to improve its performance and the user experience.

§ 5 Design overview

At a high level, this proposal consists of two parts:

- Annotate *some* existing library preconditions as *hardened*.

- Define the notion of a *hardened implementation*. In a hardened implementation, *hardened preconditions* are checked at runtime and violating them is a contract violation.

There are a few important aspects to the proposed design:

- In a *hardened implementation*, it is not possible to “turn off” these precondition checks by using the ignore semantic. Since we specify that violating a *hardened precondition* is a contract violation, we are already past the point where ignore could bypass the check.
- We only propose to formally check (some of the) existing function preconditions, not to add any new preconditions. As a drive-by, we also make a few existing preconditions more explicit in the specification (e.g. `vector::operator[]` only has implicit preconditions via iterator validity).
- Hardening deals exclusively with preconditions that must be checked at runtime. Thus, performance overhead is an important constraint.
- Hardening is intended to avoid requiring changes to an implementation’s ABI. In particular, the intent is to make it possible for implementations to allow mixing hardened and unhardened code in the same program, if they wish to do so.
- Hardening needs to be lightweight enough to be used in production environments. In our experience, a debug-only approach is not sufficient to really move the needle security-wise. Thus, the goal is to identify the minimal set of checks that would deliver the most value to users while requiring minimal overhead, with a particular focus on memory safety as a major source of security vulnerabilities. Checking *all* preconditions in the library is an explicit non-goal, and would be impossible for many semantic requirements anyway.

To reiterate the last point, an important design principle is that hardening needs to be lightweight enough for production use by a wide variety of real-world programs. In our experience in `libc++`, a small set of checks that is widely used delivers far more value than a more extensive set of checks that is only enabled by select few users. Thankfully, many of the most valuable checks, such as checking for out-of-bounds access in standard containers, also happen to be relatively cheap.

§ 6 Hardened preconditions

To specify hardening in the Standard, this proposal introduces the notion of a *hardened precondition*. A *hardened precondition* is a precondition that results in a contract violation in a *hardened implementation*. Adding hardening to the library largely consists of turning some of the existing preconditions into *hardened preconditions* in the specification. For example:

(23.7.2.2.6) **Element access** [`span.elem`]

```
constexpr reference operator[](size_type idx) const;
```

¹ **Hardened** **P**reconditions: `idx < size()` is `true`.

In the initial proposal, we decide to focus on hardened preconditions that prevent out-of-bounds memory access, i.e., compromise the memory safety of the program. These are some of the most valuable for the user since they help prevent potential security vulnerabilities; many of them are also relatively cheap to implement. More hardened preconditions can potentially be added in the future, but the intent is for their number to be limited to keep hardening viable for production use. Specifically, the proposal is to add hardened preconditions to:

- Accessors of sequence containers, `std::span`, `std::mdspan`, `std::string`, `std::string_view` and other similar classes that might attempt to access non-existent elements (e.g. `back()` on an empty container or `operator[]` with an invalid index).
- Modifiers of sequence containers and string classes that assume the container to be non-empty (e.g. `pop_back()`).
- Accessors of `optional` and `expected` that expect the object to be non-empty.

In our experience, hardening all of these operations is trivial to implement and provides significant security value.

§ 7 Enabling hardening

Much like a freestanding implementation, the way to request a *hardened* implementation is left for the implementation to define. For example, similarly to `-ffreestanding`, we expect that most toolchains would provide a compiler flag like `-fhardened`, but other alternatives like a `-D_LIBCPP_HARDENING_MODE=<mode>` macro would also be conforming. If this proposal gets accepted, we expect implementations to converge on a portable mechanism. Other details like whether hardened implementations and non-hardened implementations can be mixed in different translation units are intentionally left unspecified, to avoid overconstraining the implementations.

§ 8 Relationship to Contracts, Profiles, and Erroneous Behavior

§ 8.1 Contracts

Contracts is a new (in-progress) language feature that allows expressing preconditions and much more, with a lot of flexibility on what happens when an assertion is not satisfied. In the latest revision of this paper, we decided to base *hardened preconditions* on contract violations for several reasons:

- It provides a unifying framework for specifying what should be done when “something goes wrong” both at the language and the library level. A unified

mechanism is something that our current hardening clients have been repeatedly requesting.

- It provides flexibility for handling contract violations in a way that best suits the user. In particular, our deployment experience has made it clear that different users have different requirements, and that e.g. the weaker observe semantics can be extremely useful to allow deploying this at scale.

It is useful to note that we don't require implementations to actually implement these checks as contract assertions. Implementations are free to implement precondition checking however they see fit (e.g. a macro), however they are required to employ the contract violation handling mechanism when a precondition is not satisfied.

Note that if Contracts were to not be pursued anymore, this feature could easily be reworded in terms of a guaranteed termination in an implementation-defined way, or using Erroneous Behavior. We are not strongly attached to the exact mechanism used to implement this, but we find that Contracts is a nearly perfect fit.

§ 8.2 Profiles

The various Profiles proposals introduce a framework to specify sets of safety guarantees (such as a type safety profile or an invalidation profile). If profiles become a part of the Standard in the future, hardening can most likely be formulated as an additional profile; this would formalize how hardening is turned on and off.

However, we feel strongly that a hardening mode as specified in this paper standardizes existing practice and delivers value today without waiting for a larger and still experimental language feature.

§ 8.3 Erroneous Behavior

While Erroneous Behavior is a way to clearly mark some code as incorrect in the specification, it does not clearly specify *what* should happen in case of EB. For example, a conforming behavior for `vector::operator[]` being called out-of-bounds under EB would be to return the last element of the vector instead. While that is well-defined behavior, we feel that it is not especially useful behavior and that is certainly not what our users are looking for. In contrast, the Contracts facilities provide a well-defined and flexible framework to handle this.

§ 9 FAQ

§ 9.1 Notable omissions

§ 9.1.1 erase member functions

Most containers have an `erase` member function that takes an iterator into the container and erases the element it points to. These functions have an implicit precondition that comes from the fact that the iterator argument is defined to denote “a valid dereferenceable constant iterator to [the container]”.

Unfortunately, this precondition can only be checked for some containers and not others. For a contiguous container, the iterator can be lowered to a pointer; the pointer can then be tested to check whether it points within the bounds of the container’s underlying storage (using `std::less` and the related function object classes). There is, however, no feasible way to do a similar check for a non-contiguous container (without requiring an ABI break and significant overhead). In this paper, we would prefer to focus on preconditions that can be checked in a clear and straightforward way across all applicable classes; less obvious cases would be better served by a separate dedicated paper.

§ 9.1.2 Associative containers

Most preconditions of associative containers do not map clearly to spatial safety; moreover, associative containers have a significantly different interface and implementation compared to the containers covered by this paper. Since specifying preconditions that are relevant for memory safety in the associative containers is less straightforward than for sequence containers and since we don’t have sufficient implementation experience hardening those, we prefer to explore hardening for associative containers in a dedicated paper.

§ 9.1.3 Algorithms

All algorithms (in fact, all library functions) that operate on ranges have an implicit precondition that the ranges are valid. Unfortunately, that precondition can only be checked partially and for contiguous iterators, where it is trivial to check that the end of the range is reachable from the beginning. In the general case, it is also impossible to check that both iterators point into the same container, which doesn’t appear to be covered by the “valid range” requirement but is required in practice for these algorithms to do something sensible. Moreover, since “valid range” is a blanket requirement effectively covering all library functions, we have limited implementation experience with hardening algorithms, our current approach having focused on containers and container-like classes so far.

Some algorithms taking one range as an input and another as an output have a precondition that the two ranges do not overlap. This precondition also cannot be checked in the general case (for example, the iterators might be input iterators, meaning they cannot be “rewinded” after being incremented which would be required to perform a check).

For all of these reasons, we decided to leave hardening of the algorithms to a dedicated paper.

§ 9.1.4 `mdspan` extents and layout classes

Classes representing the extents and the layout mapping of an `mdspan` (`extents`, `layout_left`, `layout_right`, `layout_stride`) contain several preconditions that can be considered for hardening. However, these are large, complicated classes with non-trivial interactions with the `mdspan` class itself; it would require dedicated effort and research to make sure all relevant cases are hardened and there is no redundancy leading to unnecessary overhead. As such, we would prefer to keep the initial proposal focused on preconditions that are trivial to check and reason about and defer `mdspan` to a separate paper.

§ 9.1.5 `valarray`

Similarly to `mdspan`, `valarray` is non-trivial to harden properly. Its API is effectively spread across several helper classes that interact in non-trivial ways. After surveying our implementation, we believe that our implementation experience with `valarray` hardening is not sufficient at this time to include more than basic hardening of `operator[]` in this paper. We would prefer to leave a thorough exploration of `valarray` to a dedicated paper.

§ 9.2 Specifying previously implicit preconditions for sequence containers

Before this proposal, several member functions of sequence containers (specifically, `front`, `back` and `operator[]`) do not have an explicit precondition. Rather, their preconditions are formulated implicitly via a *Returns* or an *Effects* element of the function description (for example, “*Returns*: `*a.begin()`” implies that `a.begin()` returns a dereferenceable iterator). It might seem that the most natural way to express the precondition would be to assume the container is not empty (as is the case for some other functions defined in the same section, such as `pop_back`):

```
a.front()
```

⁶⁹ *Result*: reference; `const_reference` for constant `a`.

⁷⁰ *Hardened preconditions*: `!a.empty()`.

⁷¹ *Returns*: `*a.begin()`

However, this formulation would be subtly different from the existing implicit precondition. Consider an implementation that defines, let’s say, a deque iterator type such that the singular iterator returned by `a.end()` is dereferenceable and returns a reference to some sentinel value (perhaps as an attempt to be “user-friendly”). There doesn’t seem to be anything in the current text of the Standard that rules out such an implementation; thus, under the current specification, calling `a.front()` on an empty deque would be perfectly well-defined when using that implementation. However, if we were to formulate the precondition to assume that the container is not empty, that would make this previously-valid code run into undefined behavior in a non-hardened implementation (and terminate in a hardened implementation).

For this reason, we try to stay as close as possible to the existing specification and formulate the precondition in terms of iterator validity:

```
a.front()
```

69 *Result:* reference; const_reference for constant a.

70 *Hardened preconditions:* a.begin() is dereferenceable.

71 *Returns:* *a.begin()

While expressing a precondition this way might make it seem complicated, in practice an implementation would know exactly when the iterator is dereferenceable, making this check easy to implement (most implementations can simply check that the container is not empty).

§ 10 Proposed wording

§ 10.1 Introduce hardened preconditions

Add a new paragraph to 4.1 [intro.compliance] after paragraph 7 as indicated:

7 Two kinds of implementations are defined: a *hosted implementation* and a *freestanding implementation*. A freestanding implementation is one in which execution may take place without the benefit of an operating system. A hosted implementation supports all the facilities described in this document, while a freestanding implementation supports the entire C++ language described in [lex] through [cpp] and the subset of the library facilities described in [compliance].

8 Additionally, an implementation can be a *hardened implementation*. A hardened implementation is one in which violating a *hardened precondition* is a contract violation.

Add a new element to 16.3.2.4 [structure.specifications] after element 3.3 as indicated:

(3.3) *Preconditions:* the conditions that the function assumes to hold whenever it is called; violation of any preconditions results in undefined behavior.

(3.4) *Hardened preconditions:* the conditions that the function assumes to hold whenever it is called; violation of any hardened preconditions results in a contract violation in a hardened implementation, and undefined behavior otherwise.

§ 10.2 span

§ 10.2.1 span::operator[]

Modify 23.7.2.2.6 [span.elem] paragraph 1 as indicated:

- ```
constexpr reference operator[](size_type idx) const;
```
- 1 *Hardened ~~P~~preconditions:* `idx < size()` is `true`.
  - 2 *Returns:* `*(data() + idx)`.
  - 3 *Throws:* Nothing.

## § 10.2.2 span::front

Modify 23.7.2.2.6 [span.elem] paragraph 6 as indicated:

- ```
constexpr reference front() const;
```
- 6 *Hardened ~~P~~preconditions:* `empty()` is `false`.
 - 7 *Returns:* `*data()`.
 - 8 *Throws:* Nothing.

§ 10.2.3 span::back

Modify 23.7.2.2.6 [span.elem] paragraph 9 as indicated:

- ```
constexpr reference back() const;
```
- 9 *Hardened ~~P~~preconditions:* `empty()` is `false`.
  - 10 *Returns:* `*(data() + (size() - 1))`.
  - 11 *Throws:* Nothing.

## § 10.2.4 span::first

Modify 23.7.2.2.4 [span.sub] paragraph 2 as indicated:

- ```
template<size_t Count> constexpr span<element_type, Count>  
first() const;
```
- 1 *Mandates:* `Count <= Extent` is `true`.
 - 2 *Hardened ~~P~~preconditions:* `Count <= size()` is `true`.
 - 3 *Effects:* Equivalent to: `return R{data(), Count};` where R is the return type.

Modify 23.7.2.2.4 [span.sub] paragraph 11 as indicated:

```
constexpr span<element_type, dynamic_extent> first(size_type
count) const;
```

11 *Hardened ~~P~~reconditions*: count <= size() is true.

12 *Effects*: Equivalent to: return {data(), count};

§ 10.2.5 span::last

Modify 23.7.2.2.4 [span.sub] paragraph 5 as indicated:

```
template<size_t Count> constexpr span<element_type, Count>
last() const;
```

4 *Mandates*: Count <= Extent is true.

5 *Hardened ~~P~~reconditions*: Count <= size() is true.

6 *Effects*: Equivalent to: return R{data() + (size() - Count), Count};
where R is the return type.

Modify 23.7.2.2.4 [span.sub] paragraph 13 as indicated:

```
constexpr span<element_type, dynamic_extent> last(size_type
count) const;
```

13 *Hardened ~~P~~reconditions*: count <= size() is true.

14 *Effects*: Equivalent to: return {data() + (size() - count), count};

§ 10.2.6 span::subspan

Modify 23.7.2.2.4 [span.sub] paragraph 8 as indicated:

```
template<size_t Offset, size_t Count = dynamic_extent>
constexpr span<element_type, see below> subspan() const;
```

7 *Mandates*:

```
Offset <= Extent && (Count == dynamic_extent || Count <=
Extent - Offset)
```

is true.

8 *Hardened ~~P~~reconditions*:

```
Offset <= size() && (Count == dynamic_extent || Count <=
size() - Offset)
```

is true.

Modify 23.7.2.2.4 [span.sub] paragraph 15 as indicated:

```
constexpr span<element_type, dynamic_extent> subspan(
    size_type offset, size_type count = dynamic_extent) const;
```

15 *Hardened ~~P~~preconditions:*

```
offset <= size() && (count == dynamic_extent || count <=
    size() - offset)
```

is `true`.

§ 10.2.7 span constructors

Modify 23.7.2.2.2 [span.cons] around paragraph 4 as indicated:

```
template<class It>
    constexpr explicit(extent != dynamic_extent) span(It first,
        size_type count);
```

[...]

4 *Preconditions:*

(4.1) - `[first, first + count)` is a valid range.

(4.2) - It models `contiguous_iterator`.

(4.3) - If `extent` is not equal to `dynamic_extent`, then `count` is equal to `extent`.

5 *Hardened preconditions:*

(5.1) - If `extent` is not equal to `dynamic_extent`, then `count` is equal to `extent`.

? *Effects:* Initializes `data_` with `to_address(first)` and `size_` with `count`.

? *Throws:* Nothing.

Modify 23.7.2.2.2 [span.cons] around paragraph 8 as indicated:

```
template<class It, class End>
    constexpr explicit(extent != dynamic_extent) span(It first,
        End last);
```

[...]

8 *Preconditions:*

(8.1) - If `extent` is not equal to `dynamic_extent`, then `last - first` is equal to `extent`.

(8.?) - `[first, last)` is a valid range.

(8.?) - It models `contiguous_iterator`.

(8.?) - End models `sized_sentinel_for<It>`.

9 *Hardened preconditions:*

(9.1) - If `extent` is not equal to `dynamic_extent`, then `last - first` is equal to `extent`.

? *Effects:* Initializes `data_` with `to_address(first)` and `size_` with `last - first`.

? *Throws:* When and what `last - first` throws.

Modify 23.7.2.2.2 [span.cons] around paragraph 15 as indicated:

```
template<class R> constexpr explicit(extent != dynamic_extent)
    span(R&& r);
```

[...]

15 *Preconditions:*

(15.1) - If `extent` is not equal to `dynamic_extent`, then `ranges::size(r)` is equal to `extent`.

(15.?) - R models `ranges::contiguous_range` and `ranges::sized_range`.

(15.?) - If `is_const_v<element_type>` is `false`, R models `ranges::borrowed_range`.

16 *Hardened preconditions:*

(16.1) - If `extent` is not equal to `dynamic_extent`, then `ranges::size(r)` is equal to `extent`.

? *Effects:* Initializes `data_` with `ranges::data(r)` and `size_` with `ranges::size(r)`.

? *Throws:* What and when `ranges::data(r)` and `ranges::size(r)` throw.

Modify 23.7.2.2.2 [span.cons] paragraph 19 as indicated:

```
constexpr explicit(extent != dynamic_extent)
    span(std::initializer_list<value_type> il);
```

18 *Constraints:* `is_const_v<element_type>` is `true`.

19 *Hardened Preconditions:* If `extent` is not equal to `dynamic_extent`, then `il.size()` is equal to `extent`.

20 *Effects:* Initializes `data_` with `il.begin()` and `size_` with `il.size()`.

Modify 23.7.2.2.2 [span.cons] paragraph 23 as indicated:

```
template<class OtherElementType, size_t OtherExtent>
    constexpr explicit(see below) span(const
        span<OtherElementType, OtherExtent>& s) noexcept;
```

22 *Constraints:*

- (22.1) - extent == dynamic_extent || OtherExtent == dynamic_extent ||
extent == OtherExtent is true, and
- (22.2) - is_convertible_v<OtherElementType(*)[], element_type(*)[> is
true.

[Note 6: The intent is to allow only qualification conversions of the
OtherElementType to element_type. — end note]

- 23 *Hardened Ppreconditions:* If extent is not equal to dynamic_extent, then
s.size() is equal to extent.
- 24 *Effects:* Constructs a span that is a view over the range [s.data(),
s.data() + s.size()).

§ 10.3 basic_string_view

§ 10.3.1 basic_string_view::operator[]

Modify 27.3.3.6 [string.view.access] paragraph 1 as indicated:

```
constexpr const_reference operator[](size_type pos) const;
```

- 1 *Hardened Ppreconditions:* pos < size().
- 2 *Returns:* data_[pos].
- 3 *Throws:* Nothing.
- 4 [Note I: Unlike basic_string::operator[],
basic_string_view::operator[](size()) ~~has undefined behavior~~ violates
a hardened precondition instead of returning charT(). — end note]

§ 10.3.2 basic_string_view::front

Modify 27.3.3.6 [string.view.access] paragraph 7 as indicated:

```
constexpr const_reference front() const;
```

- 7 *Hardened Ppreconditions:* !empty().
- 8 *Returns:* data_[0].

9 *Throws: Nothing.*

§ 10.3.3 `basic_string_view::back`

Modify 27.3.3.6 [string.view.access] paragraph 10 as indicated:

```
constexpr const_reference back() const;
```

10 *Hardened ~~P~~preconditions: !empty().*

11 *Returns: data_[size() - 1].*

12 *Throws: Nothing.*

§ 10.3.4 `basic_string_view::remove_prefix`

Modify 27.3.3.7 [string.view.modifiers] paragraph 1 as indicated:

```
constexpr void remove_prefix(size_type n);
```

1 *Hardened ~~P~~preconditions: n <= size().*

2 *Effects: Equivalent to: data_ += n; size_ -= n;*

§ 10.3.5 `basic_string_view::remove_suffix`

Modify 27.3.3.7 [string.view.modifiers] paragraph 3 as indicated:

```
constexpr void remove_suffix(size_type n);
```

3 *Hardened ~~P~~preconditions: n <= size().*

4 *Effects: Equivalent to: size_ -= n;*

§ 10.4 Sequence containers

§ 10.4.1 `a.front()`

Modify 23.2.4 [sequence.reqmts] around paragraph 69 as indicated:

```
a.front()
```

69 *Result: reference; const_reference for constant a.*

70 *Hardened precondition: a.begin() is dereferenceable.*

71 *Returns: *a.begin()*

72 *Remarks: Required for basic_string, array, deque, forward_list, inplace_vector, list, and vector.*

[*Note ?*: In most cases, the hardened precondition is equivalent to `!a.empty()`. — *end note*]

§ 10.4.2 `a.back()`

Modify 23.2.4 [sequence.reqmts] around paragraph 72 as indicated:

`a.back()`

72 *Result*: reference; `const_reference` for constant `a`.

73 *Hardened preconditions*: `std::prev(a.end())` is dereferenceable.

74 *Effects*: Equivalent to:

```
auto tmp = a.end();
--tmp;
return *tmp;
```

75 *Remarks*: Required for `basic_string`, `array`, `deque`, `inplace_vector`, `list`, and `vector`.

[*Note ?*: In most cases, the hardened precondition is equivalent to `!a.empty()`. — *end note*]

§ 10.4.3 `a.pop_front()`

Modify 23.2.4 [sequence.reqmts] paragraph 110 as indicated:

`a.pop_front()`

109 *Result*: `void`

110 *Hardened Ppreconditions*: `a.empty()` is `false`.

111 *Effects*: Destroys the first element.

112 *Remarks*: Required for `deque`, `forward_list`, and `list`.

§ 10.4.4 `a.pop_back()`

Modify 23.2.4 [sequence.reqmts] paragraph 114 as indicated:

`a.pop_back()`

113 *Result*: `void`

114 *Hardened Ppreconditions*: `a.empty()` is `false`.

115 *Effects*: Destroys the last element.

116 *Remarks:* Required for `basic_string`, `deque`, `inplace_vector`, `list`, and `vector`.

§ 10.4.5 `a[n]`

Modify 23.2.4 [sequence.reqmts] around paragraph 117 as indicated:

`a[n]`

117 *Result:* reference; `const_reference` for constant `a`

118 *Hardened preconditions:* `a.begin() + n` is dereferenceable.

119 *Effects:* Equivalent to: `return *(a.begin() + n);`

120 *Remarks:* Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

[Note ?: In most cases, the hardened precondition is equivalent to `n < a.size()`. — end note]

§ 10.5 `basic_string`

§ 10.5.1 `basic_string::operator[]`

Modify 27.4.3.8.1 [string.accessors] paragraph 1 as indicated:

```
constexpr const_reference operator[](size_type pos) const;
constexpr reference      operator[](size_type pos);
```

1 *Hardened ~~P~~preconditions:* `pos <= size()`.

2 *Returns:* `*(begin() + pos)` if `pos < size()`. Otherwise, returns a reference to an object of type `charT` with value `charT()`, where modifying the object to any value other than `charT()` leads to undefined behavior.

3 *Throws:* Nothing.

4 *Complexity:* Constant time.

§ 10.5.2 `basic_string::front`

Modify 27.4.3.8.1 [string.accessors] paragraph 7 as indicated:

```
constexpr const charT& front() const;
constexpr charT& front();
```

7 *Hardened ~~P~~preconditions:* `!empty()`.

8 *Effects:* Equivalent to: `return operator[](0);`

§ 10.5.3 `basic_string::back`

Modify 27.4.3.8.1 [[string.accessors](#)] paragraph 9 as indicated:

```
constexpr const charT& back() const;
constexpr charT& back();
```

9 *Hardened P*preconditions: `!empty()`.

10 *Effects*: Equivalent to: `return operator[](size() - 1);`

§ 10.5.4 `basic_string::pop_back`

Modify 27.4.3.7.5 [[string.erase](#)] paragraph 12 as indicated:

```
constexpr void pop_back();
```

12 *Hardened P*preconditions: `!empty()`.

13 *Effects*: Equivalent to `erase(end() - 1)`.

14 *Throws*: Nothing.

§ 10.6 `mdspan`

§ 10.6.1 `mdspan::operator[]`

Modify 23.7.3.6.3 [[mdspan.mdspan.members](#)] paragraph 3 as indicated:

```
template<class... OtherIndexTypes>
    constexpr reference operator[](OtherIndexTypes... indices)
        const;
```

[...]

2 Let `I` be `extents_type::index-cast(std::move(indices))`.

3 *Hardened P*preconditions: `I` is a multidimensional index in `extents()`.

[*Note 1*: This implies that `map_(I) < map_.required_span_size()` is `true`.
— end note]

§ 10.6.2 `mdspan` constructor

Modify 23.7.3.6.2 [[mdspan.mdspan.cons](#)] around paragraph 21 as indicated:

```
template<class OtherElementType, class OtherExtents,
         class OtherLayoutPolicy, class OtherAccessor>
    constexpr explicit(see below)
        mdspan(const mdspan<OtherElementType, OtherExtents,
                           OtherLayoutPolicy, OtherAccessor>&
               other);
```

[...]

21 *Preconditions:*

(21.1) - For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is true.

(21.2) - `[0, map_.required_span_size())` is an accessible range of `ptr_` and `acc_` for values of `ptr_`, `map_`, and `acc_` after the invocation of this constructor.

22 *Hardened preconditions:*

(22.1) - For each rank index `r` of `extents_type`, `static_extent(r) == dynamic_extent || static_extent(r) == other.extent(r)` is true.

§ 10.7 bitset

§ 10.7.1 bitset::operator[]

Modify 22.9.2.3 [bitset.members] paragraph 30 as indicated:

```
constexpr bool operator[](size_t pos) const;
```

30 *Hardened Ppreconditions:* `pos` is valid.

31 *Returns:* `true` if the bit at position `pos` in `*this` has the value one, otherwise `false`.

32 *Throws:* Nothing.

Modify 22.9.2.3 [bitset.members] paragraph 33 as indicated:

```
constexpr bitset::reference operator[](size_t pos);
```

33 *Hardened Ppreconditions:* `pos` is valid.

34 *Returns:* An object of type `bitset::reference` such that `(*this)[pos] == this->test(pos)`, and such that `(*this)[pos] = val` is equivalent to `this->set(pos, val)`.

35 *Throws:* Nothing.

§ 10.8 valarray

§ 10.8.1 valarray::operator[]

Modify 29.6.2.4 [valarray.access] paragraph 1 as indicated:

```
const T& operator[](size_t n) const;  
T& operator[](size_t n);
```

- 1 *Hardened P*preconditions: `n < size()` is `true`.
- 2 *Returns*: A reference to the corresponding element of the array.

§ 10.9 optional

§ 10.9.1 optional::operator->

Modify 22.5.3.7 [optional.observe] paragraph 1 as indicated:

```
constexpr const T* operator->() const noexcept;  
  
constexpr T* operator->() noexcept;
```

- 1 *Hardened P*preconditions: `*this` contains a value.
- 2 *Returns*: `*val`.
- 3 *Remarks*: These functions are `constexpr` functions.

§ 10.9.2 optional::operator*

Modify 22.5.3.7 [optional.observe] paragraph 4 as indicated:

```
constexpr const T& operator*() const & noexcept;  
  
constexpr T& operator*() & noexcept;
```

- 4 *Hardened P*preconditions: `*this` contains a value.
- 5 *Returns*: `*val`.
- 6 *Remarks*: These functions are `constexpr` functions.

Modify 22.5.3.7 [optional.observe] paragraph 7 as indicated:

```
constexpr T&& operator*() && noexcept;  
  
constexpr const T&& operator*() const && noexcept;
```

- 7 *Hardened P*preconditions: `*this` contains a value.
- 8 *Effects*: Equivalent to: `return std::move(*val);`

§ 10.10 expected

§ 10.10.1 expected::operator->

Modify 22.8.6.6 [expected.object.obs] paragraph 1 as indicated:

```
constexpr const T* operator->() const noexcept;
```

```
constexpr T* operator->() noexcept;
```

1 *Hardened Ppreconditions*: has_value() is true.

2 *Returns*: addressof(val).

§ 10.10.2 expected::operator*

Modify 22.8.6.6 [expected.object.obs] paragraph 3 as indicated:

```
constexpr const T& operator*() const & noexcept;
```

```
constexpr T& operator*() & noexcept;
```

3 *Hardened Ppreconditions*: has_value() is true.

4 *Returns*: val.

Modify 22.8.6.6 [expected.object.obs] paragraph 5 as indicated:

```
constexpr T&& operator*() && noexcept;
```

```
constexpr const T&& operator*() const && noexcept;
```

5 *Hardened Ppreconditions*: has_value() is true.

6 *Returns*: std::move(val).

§ 10.10.3 expected::error()

Modify 22.8.6.6 [expected.object.obs] paragraph 14 as indicated:

```
constexpr const E& error() const & noexcept;
```

```
constexpr E& error() & noexcept;
```

14 *Hardened Ppreconditions*: has_value() is false.

15 *Returns*: unex.

Modify 22.8.6.6 [expected.object.obs] paragraph 16 as indicated:

```
constexpr E&& error() && noexcept;
```

```
constexpr const E&& error() const && noexcept;
```

16 *Hardened Ppreconditions*: has_value() is false.

17 *Returns*: std::move(unex).

§ 10.11 Feature-test macros

(Authors' note: we are not attached to any particular way of defining the feature-test macros. The proposed wording adds a macro for each class that would now contain hardened preconditions, but we are very open to suggestions on how these macros may be combined or split further, as well as naming)

Modify 17.3.2 [version.syn] as indicated:

```
#define __cpp_lib_generator 202207L // also in
    <generator>
#define __cpp_lib_generic_associative_lookup 201304L // also in
    <map>, <set>
#define __cpp_lib_generic_unordered_lookup 201811L // also in
    <unordered_map>, <unordered_set>
+ #define __cpp_lib_hardened_array 20????L // also in
    <array>
+ #define __cpp_lib_hardened_basic_string 20????L // also in
    <string>
+ #define __cpp_lib_hardened_basic_string_view 20????L // also in
    <string_view>
+ #define __cpp_lib_hardened_bitset 20????L // also in
    <bitset>
+ #define __cpp_lib_hardened_deque 20????L // also in
    <deque>
+ #define __cpp_lib_hardened_expected 20????L // also in
    <expected>
+ #define __cpp_lib_hardened_forward_list 20????L // also in
    <forward_list>
+ #define __cpp_lib_hardened_inplace_vector 20????L // also in
    <inplace_vector>
+ #define __cpp_lib_hardened_list 20????L // also in
    <list>
+ #define __cpp_lib_hardened_mdspan 20????L // also in
    <mdspan>
+ #define __cpp_lib_hardened_optional 20????L // also in
    <optional>
+ #define __cpp_lib_hardened_span 20????L // also in
    <span>
+ #define __cpp_lib_hardened_valarray 20????L // also in
    <valarray>
+ #define __cpp_lib_hardened_vector 20????L // also in
    <vector>
    #define __cpp_lib_hardware_interference_size 201703L //
        freestanding, also in <new>
    #define __cpp_lib_has_unique_object_representations 201606L //
        freestanding, also in <type_traits>
#define __cpp_lib_hazard_pointer 202306L // also in
    <hazard_pointer>
#define __cpp_lib_hypot 201603L // also in
    <cmath>
```